email-rules

Release 0.0.0

Henry Asa

Apr 30, 2023

CONTENTS:

1 API Reference **1**

1.1 gmail_rules.actions.build_xmls . . . 1

1.2 gmail_rules.actions.rule_collection . . . 1

1.3 gmail_rules.rules.rule . . . 3

1.4 gmail_rules.rules.copy_to . . . 6

1.5 gmail_rules.rules.move_to . . . 8

1.6 gmail_rules.utils.helpers . . . 11

2 Indices and Tables **13**

Python Module Index **15**

Index **17**

CHAPTER

ONE

API REFERENCE

actions.build_xmls
actions.rule_collection
rules.rule
rules.copy_to
rules.move_to
utils.helpers

1.1 gmail_rules.actions.build_xmls

Functions

build_xml_text(text)	Build a final string that can be pasted into a .xml file from the strings returned by the *build_rule()* methods in the *Rule* classes

build_xml_text(*text: str*) → str

Build a final string that can be pasted into a .xml file from the strings returned by the *build_rule()* methods in the *Rule* classes

1.2 gmail_rules.actions.rule_collection

Functions

set_module(module)	Private decorator for overriding __module__ on a function or class.

Classes

Rule_Collection([name])	Collection of `Rule` that can be organized and stored together

class Rule_Collection(*name: str = 'Rule Collection'*)

Bases: `object`

Collection of `Rule` that can be organized and stored together

Parameters
name (`str`, optional) – Name of this collection of rules

add_rule(*rule_to_add: Rule*) → None

Alias for *Rule_Collection.add_rules()*. Adds `Rule` to a *Rule_Collection*

Parameters
rules_to_add (*list or tuple or set or frozenset or dict*) – The `Rule` that should be added to the *Rule_Collection*

Raises

- **KeyError** – Raises a *KeyError* when a certain rule is already in the collection
- **TypeError** – Raises a *TypeError* when a rule is not of type `Rule`
- **TypeError** – Raises a *TypeError* if a rule is not an iterable of `Rule`

add_rules(*rules_to_add: Rule | list | tuple | set | frozenset | dict*) → None

Add `Rule` objects to a *Rule_Collection*

Parameters
rules_to_add (`Rule` or list or tuple or set or frozenset or dict) – The `Rule` (or `Rule`) that should be added to the *Rule_Collection*

Raises

- **KeyError** – Raises a *KeyError* when a certain rule is already in the collection
- **TypeError** – Raises a *TypeError* when a rule is not of type `Rule`
- **TypeError** – Raises a *TypeError* if a rule is not an iterable of `Rule`

build_final_string(*additional_comment: str = None*) → str

Builds the final properly formatted collection of rules

Parameters
additional_comment (*str, optional*) – Adds a final comment above the entire rule string, by default *None*

Returns
final string representing all of the `Rule` in the collection

Return type
str

property final_string: str

Generates the final string representation of a collection of rules

Returns
str representing the xmls of all the rules in this collection

Return type
str

1.3 gmail_rules.rules.rule

Functions

`set_module`(module)	Private decorator for overriding __module__ on a function or class.

Classes

Rule([list_of_emails, rule_defaults, rule_name])	Defines an individual mail rule and its necessary attributes

class Rule(*list_of_emails: list = [], rule_defaults: dict = {}, rule_name: str = 'Mail Filter'*)

Bases: `object`

Defines an individual mail rule and its necessary attributes

A rule class defines an individual mail rule and includes functions/operations to modify a rule's behavior and generate parseable formats of a rule.

...

name

Name of the mail rule.

Type
str

final_rule

String representing the final formatted rule

Type
str

emails_list

List of email addresses (`str`) that the mail rule applies to

Type
list

concatenated_emails

String of email addresses that are concatenated together

Type
str

rule_header

Generic string that precedes the unique section of mail rules

Type
str

rule_footer

Generic string that appends the unique section of mail rules

Type
str

add_attribute(*name: str*, *value: str*, *is_custom_attribute: bool = False*) → None

Add an attribute to the mail rule

Parameters

- **name** (*str*) – Name of the attribute to add
- **value** (*str*) – Value of the attribute
- **is_custom_attribute** (*bool, optional*) – Defines whether the attribute being added is custom (use with caution), by default *False*. Use this with caution, as the mail rule interpreter may not be able to parse a rule with a custom attribute

Raises

- **KeyError** – Raises a *KeyError* when an attribute has already been defined for this rule
- **KeyError** – Raises a *KeyError* if an attribute is not valid

add_attributes(*attributes_to_add: dict*) → None

Add multiple attributes to a *Rule*

Parameters
attributes_to_add (*dict*) – Dictionary where key is the name of the attribute to add (*str*) and its value (*str*) is the value of the attribute to add corresponding to that key

Raises
TypeError – Raises a *TypeError* if *attributes_to_add* is not a dictionary

add_label(*label: str*) → None

Alias for *Rule.add_label()*. Adds labels to the mail rule

Parameters
labels (*list | tuple | set | frozenset | dict*) – The label (or labels) to be added to the rule

Raises
TypeError – Raises a *TypeError* if the label is not a valid type

add_labels(*labels: str | list | tuple | set | frozenset | dict*) → None

Adds labels to the mail rule

Parameters
labels (*list | tuple | set | frozenset | dict*) – The label (or labels) to be added to the rule

Raises
TypeError – Raises a *TypeError* if the label is not a valid type

build_rule() → str

After all of the details of a rule are defined, this function is run to actually build the desired mail rule. It takes an optional argument *rule_name* which is a *str* representing the name of the mail rule, but when the rule is parsed into Gmail, this gets ignored.

concatenate(*elements_input: list*, *separator: str = ' OR '*) → str

Given a list of elements *elements*, this returns the concatenation of the elements with a separator *separator* between them (*" OR "* by default)

Parameters

- **elements_input** (*list*) – This is a list of items that should be concatenated together
- **separator** (*str*, default = *" OR "*, optional) – This is the string that will be used to separate individual elements, by default *" OR "*

Returns

final_output – Returns a singular string containing all of the items in *elements* after being concatenated and separated with the *separator*

Return type

str

property final_rule_str: str

This is the final *str* that can be copied and pasted into an xml to define the rule

Returns

str representing the entire rule in xml format

Return type

str

flatten_list(*list_to_flatten: list*) → list

Converts a list of lists into a single flat list

This function takes a *list* (of potentially nested lists) and recursively flattens the list so that it is just a single list of elements that are not of type *list*

Parameters

list_to_flatten (*list*) – Input *list* (or list of lists) to be flattened

Returns

Returns a final flat list that does not contain any nested lists

Return type

list

property rule_attributes_xmls: dict

Converts *dict* of rule attributes into *dict* of attributes where keys are in xml format

Returns

rule_attribute_xmls – Dictionary where keys are the attribute and values are the xml representation of the attribute

Return type

dict

property rule_attributes_xmls_str: str

Converts `dict` of rule attributes into ordered *str* for use in final xml

Returns

rule_attribute_xmls_str – `str` representing this `Rule` as an xml

Return type

str

xml_format_rule_attribute(*name: str*, *value: str*) → str

Given an attribute name, its value, and an optional boolean determining whether this new attriute should be added to the list of attributes, this function properly builds and formats (and assigns) the defined attribute.

Parameters

- **name** (*str*) – This is the name of the attribute to define (defined by Google's docs)
- **value** (*str*) – This is the value of the desired attribute

Returns
rule_line – Returns the properly formatted attribute

Return type
str

1.4 gmail_rules.rules.copy_to

Functions

set_module(module)	Private decorator for overriding __module__ on a function or class.

Classes

Copy_To(rule_label[, list_of_emails, ...])	*Copy_To* rule object which is a sub-class of Rule

class Copy_To(*rule_label: str | list*, *list_of_emails: list = []*, *rule_defaults: dict = {}*, *rule_name: str = ''*)

Bases: *Rule*

Copy_To rule object which is a sub-class of Rule

add_attribute(*name: str*, *value: str*, *is_custom_attribute: bool = False*) → None

Add an attribute to the mail rule

Parameters

- **name** (*str*) – Name of the attribute to add
- **value** (*str*) – Value of the attribute
- **is_custom_attribute** (*bool, optional*) – Defines whether the attribute being added is custom (use with caution), by default *False*. Use this with caution, as the mail rule interpreter may not be able to parse a rule with a custom attribute

Raises

- **KeyError** – Raises a *KeyError* when an attribute has already been defined for this rule
- **KeyError** – Raises a *KeyError* if an attribute is not valid

add_attributes(*attributes_to_add: dict*) → None

Add multiple attributes to a *Rule*

Parameters

attributes_to_add (*dict*) – Dictionary where key is the name of the attribute to add (*str*) and its value (*str*) is the value of the attribute to add corresponding to that key

Raises

TypeError – Raises a *TypeError* if *attributes_to_add* is not a dictionary

add_label(*label: str*) → None

Alias for `Rule.add_label()`. Adds labels to the mail rule

Parameters

labels (*list | tuple | set | frozenset | dict*) – The label (or labels) to be added to the rule

Raises

TypeError – Raises a *TypeError* if the label is not a valid type

add_labels(*labels: str | list | tuple | set | frozenset | dict*) → None

Adds labels to the mail rule

Parameters

labels (*list | tuple | set | frozenset | dict*) – The label (or labels) to be added to the rule

Raises

TypeError – Raises a *TypeError* if the label is not a valid type

build_rule() → str

After all of the details of a rule are defined, this function is run to actually build the desired mail rule. It takes an optional argument *rule_name* which is a *str* representing the name of the mail rule, but when the rule is parsed into Gmail, this gets ignored.

concatenate(*elements_input: list, separator: str = ' OR '*) → str

Given a list of elements *elements*, this returns the concatenation of the elements with a separator *separator* between them (*" OR "* by default)

Parameters

- **elements_input** (*list*) – This is a list of items that should be concatenated together
- **separator** (*str*, default = *" OR "*, optional) – This is the string that will be used to separate individual elements, by default *" OR "*

Returns

final_output – Returns a singular string containing all of the items in *elements* after being concatenated and separated with the *separator*

Return type

str

property final_rule_str: str

This is the final *str* that can be copied and pasted into an xml to define the rule

Returns

str representing the entire rule in xml format

Return type

str

flatten_list(*list_to_flatten: list*) → list

Converts a list of lists into a single flat list

This function takes a *list* (of potentially nested lists) and recursively flattens the list so that it is just a single list of elements that are not of type *list*

Parameters
list_to_flatten (*list*) – Input *list* (or list of lists) to be flattened

Returns
Returns a final flat list that does not contain any nested lists

Return type
list

property rule_attributes_xmls: dict

Converts *dict* of rule attributes into *dict* of attributes where keys are in xml format

Returns
rule_attribute_xmls – Dictionary where keys are the attribute and values are the xml representation of the attribute

Return type
dict

property rule_attributes_xmls_str: str

Converts `dict` of rule attributes into ordered *str* for use in final xml

Returns
rule_attribute_xmls_str – `str` representing this `Rule` as an xml

Return type
str

xml_format_rule_attribute(*name: str*, *value: str*) → str

Given an attribute name, its value, and an optional boolean determining whether this new attriute should be added to the list of attributes, this function properly builds and formats (and assigns) the defined attribute.

Parameters

- **name** (*str*) – This is the name of the attribute to define (defined by Google's docs)
- **value** (*str*) – This is the value of the desired attribute

Returns
rule_line – Returns the properly formatted attribute

Return type
str

1.5 gmail_rules.rules.move_to

Functions

`set_module`(module)	Private decorator for overriding __module__ on a function or class.

Classes

Move_To(rule_label[, list_of_emails, ...])	*Move_To* rule object which is a sub-class of `Rule`

class Move_To(*rule_label: str | list, list_of_emails: list = [], rule_defaults: dict = {}, rule_name: str = ''*)

Bases: *Rule*

Move_To rule object which is a sub-class of `Rule`

add_attribute(*name: str, value: str, is_custom_attribute: bool = False*) → None

Add an attribute to the mail rule

Parameters

- **name** (*str*) – Name of the attribute to add
- **value** (*str*) – Value of the attribute
- **is_custom_attribute** (*bool, optional*) – Defines whether the attribute being added is custom (use with caution), by default *False*. Use this with caution, as the mail rule interpreter may not be able to parse a rule with a custom attribute

Raises

- **KeyError** – Raises a *KeyError* when an attribute has already been defined for this rule
- **KeyError** – Raises a *KeyError* if an attribute is not valid

add_attributes(*attributes_to_add: dict*) → None

Add multiple attributes to a *Rule*

Parameters

attributes_to_add (*dict*) – Dictionary where key is the name of the attribute to add (*str*) and its value (*str*) is the value of the attribute to add corresponding to that key

Raises

TypeError – Raises a *TypeError* if *attributes_to_add* is not a dictionary

add_label(*label: str*) → None

Alias for `Rule.add_label()`. Adds labels to the mail rule

Parameters

labels (*list | tuple | set | frozenset | dict*) – The label (or labels) to be added to the rule

Raises

TypeError – Raises a *TypeError* if the label is not a valid type

add_labels(*labels: str | list | tuple | set | frozenset | dict*) → None

Adds labels to the mail rule

Parameters

labels (*list | tuple | set | frozenset | dict*) – The label (or labels) to be added to the rule

Raises

TypeError – Raises a *TypeError* if the label is not a valid type

build_rule() → str

After all of the details of a rule are defined, this function is run to actually build the desired mail rule. It takes an optional argument *rule_name* which is a *str* representing the name of the mail rule, but when the rule is parsed into Gmail, this gets ignored.

concatenate(*elements_input: list*, *separator: str = ' OR '*) → str

Given a list of elements *elements*, this returns the concatenation of the elements with a separator *separator* between them (*" OR "* by default)

Parameters

- **elements_input** (*list*) – This is a list of items that should be concatenated together
- **separator** (*str*, default = *" OR "*, optional) – This is the string that will be used to separate individual elements, by default *" OR "*

Returns

final_output – Returns a singular string containing all of the items in *elements* after being concatenated and separated with the *separator*

Return type

str

property final_rule_str: str

This is the final *str* that can be copied and pasted into an xml to define the rule

Returns

str representing the entire rule in xml format

Return type

str

flatten_list(*list_to_flatten: list*) → list

Converts a list of lists into a single flat list

This function takes a *list* (of potentially nested lists) and recursively flattens the list so that it is just a single list of elements that are not of type *list*

Parameters

list_to_flatten (*list*) – Input *list* (or list of lists) to be flattened

Returns

Returns a final flat list that does not contain any nested lists

Return type

list

property rule_attributes_xmls: dict

Converts *dict* of rule attributes into *dict* of attributes where keys are in xml format

Returns

rule_attribute_xmls – Dictionary where keys are the attribute and values are the xml representation of the attribute

Return type

dict

property rule_attributes_xmls_str: str

Converts `dict` of rule attributes into ordered *str* for use in final xml

Returns

rule_attribute_xmls_str – `str` representing this `Rule` as an xml

Return type
str

`xml_format_rule_attribute`(*name: str*, *value: str*) → str

Given an attribute name, its value, and an optional boolean determining whether this new attriute should be added to the list of attributes, this function properly builds and formats (and assigns) the defined attribute.

Parameters

- **name** (*str*) – This is the name of the attribute to define (defined by Google's docs)
- **value** (*str*) – This is the value of the desired attribute

Returns
rule_line – Returns the properly formatted attribute

Return type
str

1.6 gmail_rules.utils.helpers

Module Attributes

TAB_SPACING	Default amount of spaces used instead of a tab (*"t"*)
ITERABLE_DATA_TYPES	Iterable data types that can store multiple instances of other objects

Functions

add_xml_comment(comment_text)	Add an xml comment to a string (does not include for newlines *"n"*)
convert_to_parseable_string(string_to_parse)	Converts a rich-text string into a parseable string containing tabs and newlines
indent(multiline_text[, amount, ...])	Indents a string with *amount indent_character*

`ITERABLE_DATA_TYPES: set = {<class 'dict'>, <class 'set'>, <class 'frozenset'>, <class 'list'>, <class 'tuple'>}`

Iterable data types that can store multiple instances of other objects

Contains: *list*, *tuple*, *set*, *frozenset*, *dict*

`TAB_SPACING: int = 4`

Default amount of spaces used instead of a tab (*"t"*)

`add_xml_comment`(*comment_text: str*) → str

Add an xml comment to a string (does not include for newlines *"n"*)

Parameters
comment_text (*str*) – Text that will be added as a comment

Returns
xml comment of *comment_text*

Return type
str

`convert_to_parseable_string`(*string_to_parse: str*) → str

Converts a rich-text string into a parseable string containing tabs and newlines

This function takes a string and allows newlines/tabs to be displayed as *n* and *t* rather than a newline and a tab

Parameters
string_to_parse (*str*) – String to display parsed richtext operations for

Returns
String displaying *n* and *t* rather than newlines and tabs

Return type
str

`indent`(*multiline_text: str*, *amount: int = 1*, *indent_character: str = '\t'*) → str

Indents a string with *amount indent_character*

Prefix each newline (*"n"*) in multiline_text with *amount* * *indent_character*

Parameters

- **multiline_text** (*str*) – Multiline str containing *"n"* that should be indented
- **amount** (*int, optional*) – The amount that *multiline_text* should be indented by, defaulted to 1
- **indent_character** (*str, optional*) – The character that *multiline_text* should be indented with, defaulted to *"t"*

Returns
The texted indented with *amount* instances of the *indent_character*

Return type
str

CHAPTER

TWO

INDICES AND TABLES

- genindex
- modindex
- search

PYTHON MODULE INDEX

g

gmail_rules.actions.build_xmls, 1
gmail_rules.actions.rule_collection, 1
gmail_rules.rules.copy_to, 6
gmail_rules.rules.move_to, 8
gmail_rules.rules.rule, 3
gmail_rules.utils.helpers, 11

INDEX

A

add_attribute() (*Copy_To method*), 6
add_attribute() (*Move_To method*), 9
add_attribute() (*Rule method*), 4
add_attributes() (*Copy_To method*), 6
add_attributes() (*Move_To method*), 9
add_attributes() (*Rule method*), 4
add_label() (*Copy_To method*), 7
add_label() (*Move_To method*), 9
add_label() (*Rule method*), 4
add_labels() (*Copy_To method*), 7
add_labels() (*Move_To method*), 9
add_labels() (*Rule method*), 4
add_rule() (*Rule_Collection method*), 2
add_rules() (*Rule_Collection method*), 2
add_xml_comment() (*in module gmail_rules.utils.helpers*), 11

B

build_final_string() (*Rule_Collection method*), 2
build_rule() (*Copy_To method*), 7
build_rule() (*Move_To method*), 9
build_rule() (*Rule method*), 4
build_xml_text() (*in module gmail_rules.actions.build_xmls*), 1

C

concatenate() (*Copy_To method*), 7
concatenate() (*Move_To method*), 10
concatenate() (*Rule method*), 4
concatenated_emails (*Rule attribute*), 3
convert_to_parseable_string() (*in module gmail_rules.utils.helpers*), 12
Copy_To (*class in gmail_rules.rules.copy_to*), 6

E

emails_list (*Rule attribute*), 3

F

final_rule (*Rule attribute*), 3
final_rule_str (*Copy_To property*), 7
final_rule_str (*Move_To property*), 10
final_rule_str (*Rule property*), 5
final_string (*Rule_Collection property*), 2
flatten_list() (*Copy_To method*), 7
flatten_list() (*Move_To method*), 10
flatten_list() (*Rule method*), 5

G

gmail_rules.actions.build_xmls
    module, 1
gmail_rules.actions.rule_collection
    module, 1
gmail_rules.rules.copy_to
    module, 6
gmail_rules.rules.move_to
    module, 8
gmail_rules.rules.rule
    module, 3
gmail_rules.utils.helpers
    module, 11

I

indent() (*in module gmail_rules.utils.helpers*), 12
ITERABLE_DATA_TYPES (*in module gmail_rules.utils.helpers*), 11

M

module
    gmail_rules.actions.build_xmls, 1
    gmail_rules.actions.rule_collection, 1
    gmail_rules.rules.copy_to, 6
    gmail_rules.rules.move_to, 8
    gmail_rules.rules.rule, 3
    gmail_rules.utils.helpers, 11
Move_To (*class in gmail_rules.rules.move_to*), 9

N

name (*Rule attribute*), 3

R

Rule (*class in gmail_rules.rules.rule*), 3
rule_attributes_xmls (*Copy_To property*), 8

rule_attributes_xmls (*Move_To property*), 10
rule_attributes_xmls (*Rule property*), 5
rule_attributes_xmls_str (*Copy_To property*), 8
rule_attributes_xmls_str (*Move_To property*), 10
rule_attributes_xmls_str (*Rule property*), 5
Rule_Collection (*class in gmail_rules.actions.rule_collection*), 2
rule_footer (*Rule attribute*), 3
rule_header (*Rule attribute*), 3

T

TAB_SPACING (*in module gmail_rules.utils.helpers*), 11

X

xml_format_rule_attribute() (*Copy_To method*), 8
xml_format_rule_attribute() (*Move_To method*), 11
xml_format_rule_attribute() (*Rule method*), 5